OTPAC A5 (m = 4): test results

Protocol: A5 with Rules R1–R9 and R11. Rule 10 (shared landing slots) was removed. Rule 11 was added: a party never predicts another party into its own next pad.

Convention: d is the maximum number of messages undelivered to a party at each of its sends, and it is enforced in every run.

What every run counts: a run that would reuse a pad stops there, and its remaining pads count as wasted.

1. Main test suite

Setup. 35,964 runs, made up of:

- 9 delivery modes;
- 12 sender orders;
- n from 16 to 2048, and d from 1 up to $\lfloor n/4 \rfloor - 1$.

Tier	n	d	Runs	Runs with reuse
A (deep)	256	2	10,800	52 (0.48%)
B (breadth)	16 – 512	1 – $\lfloor n/4 \rfloor - 1$	21,384	40 (0.19%)
C (large n)	1000 – 2048	2, 8	540	10 (1.85%)
D (large d)	64 – 256	n/8 – $\lfloor n/4 \rfloor - 1$	3,240	0
Total			**35,964**	**102 (0.28%)**

Compared with earlier versions of the rules:

Rules	Runs with reuse
R1–R10	120 (0.33%)
R1–R9	108 (0.30%)
R1–R9 + R11 (current)	**102 (0.28%)**

Run by run against R1–R10, the current rules fix 77 runs and introduce 59 new reuses.

Checks. All of these held in every run:

- the bound of d undelivered messages was never broken;
- no party ever picked a pad reserved in its own view;
- every run terminated.

Rule R8(b) inferences. R8(b) marks the pads a party must have used for messages that haven't arrived yet:

- **Exact inferences:** 2,186,867 marks, of which 1 was wrong.
- **Precautionary marks:** 48,944, of which 30,216 had really been used.

Reuse by delivery mode. 3,996 runs per mode.

Mode	instant	worst	random	adversarial	lifo	randcap	shuffle	target-sender	target-recv
Runs with reuse	0	37	7	20	0	30	8	0	0

Reuse by sender order.

Sender order	Runs with reuse
greedy-gap	53
skew	15
arena-skew	11
silent1	11
silent2	4
burst	3
stale-max	3
random	2
rr, flood, flood-rev, pair-alt	0

What the 102 reuses have in common.

- In all 102, the contested pad was still undelivered to the second user.
- In 83, the first user had reached that pad by a jump.
- None involve out-of-order delivery.

Efficiency. Efficiency = pads used / n. The mean over all runs is 0.843, against 0.839 with R1–R10.

n	d	Runs	Mean	Worst run, synchronous	Worst run, any delivery
64	2	324	0.877	0.734	0.734
128	2	324	0.932	0.875	0.805
256	2	10,800	0.963	0.867	0.699
512	2	324	0.979	0.955	0.844
1024	2	90	0.989	0.977	0.970
2048	2	90	0.993	0.994	0.810
128	8	324	0.858	0.750	0.625
512	8	324	0.954	0.877	0.773
2048	8	90	0.983	0.958	0.880
256	63	360	0.589	0.492	0.492

2. Professor Di Crescenzo's schedules

These tests ran on the previous rules (R1–R10). With the current rules:

- the example at the end of this report no longer reuses a pad;
- the all-send-orders search at n = 24, d = 1 finds 492 reusing executions instead of 564.

(1) All possible send sequences, small n and d

Every send order was explored exhaustively. For worst and lifo the delivery is maximum delay, releasing the oldest or the newest message respectively; instant is synchronous delivery.

n	d	Reuse, worst	Reuse, lifo	Reuse, instant	Exact worst-case efficiency (worst / lifo / instant)
16	1	0	0	0	0.688 / 0.625 / 0.625
20	1	0	0	0	0.750 / 0.700 / 0.700
24	1	564 of 694,329 states	0	0	0.708 / 0.708 / 0.750
28	1	1,612 of 1,990,467 states	0	0	0.679 / 0.714 / 0.714
32	1	(too large)	(too large)	0	– / – / 0.750
16 – 28	2	0	0	0	0.562 – 0.750 (worst)
32	2	0	–	–	0.781 (worst)

- **Where jumps start.** For n ≤ 20 at d = 1, no jump is possible, since a landing needs more than 4d+3 consecutive free pads. The first reuse appears at n = 24.
- **Every delivery order as well.** Varying every delivery order on top of every send order is feasible only at n = 8: 4,479 states, no reuse.

(2) Party x sends with probability p_x

Why 286 vectors and not 11^3. p_1, p_2 and p_3 each range over 0, 0.1, …, 1, which gives 11^3 = 1,331 combinations. But p_4 = 1 − (p_1 + p_2 + p_3) must not be negative. That leaves the ways of splitting 10 tenths among 4 parties: C(13,3) = **286** vectors.

Setup. Each vector was run under all 9 delivery modes.

n	d	Runs	Runs with reuse	Efficiency min / mean
32	1	7,722	2	0.563 / 0.785
64	1	7,722	12	0.688 / 0.880
64	2	7,722	34	0.625 / 0.853
128	2	7,722	41	0.727 / 0.916
256	2	5,148	37	0.797 / 0.952
256	8	2,574	13	0.668 / 0.867
512	2	2,574	29	0.867 / 0.972

Total		**41,184**	**168 (0.41%)**	

Which vectors and delivery modes give reuse.

- **Vectors.** 122 of the 286 vectors give reuse. None of them has a single sender.
- **Delivery modes.**
  - worst: 76
  - randcap: 41
  - adversarial: 33
  - shuffle: 12
  - random: 6
  - instant, lifo, target-sender, target-recv: 0

(3) Schedule (2) until near-full or parties close, then all sequences

Switch point. Schedule (2) runs until one of two conditions holds:

- (3a) at least 85% of the pad is used;
- (3b) a party has another party ahead of it within 2d+2 pads.

After the switch. From that exact state, the test explores every continuation within a window of the next few sends:

- n = 64: every delivery order for the next 4 sends, and every send order for the next 8;
- n = 128: the same for the next 3 and 6 sends.

Prefixes. The prefixes use the 0.2-step grid: 52 vectors with at least two senders, each run under random and worst delivery, giving 104 prefixes.

n	d	Trigger	Prefixes that reached the switch point	Switch states with a reusing continuation
64	1	3a	64 / 104	0
64	1	3b	104 / 104	29
128	2	3a	101 / 104	3
128	2	3b	104 / 104	0

Example of a reuse (n = 24, d = 1, delivery `worst`)

Legend:

- _ = free, x = used, A B C D = current positions;
- A and C move right; B and D move left.

```
pad       123456789012345678901234
start     A__________BC__________D
step 9    _xxxxxxxxA_BC__________D   A walks to 10
step 10   _xxxxxxxxx_BC_____A____D   A jumps to 19
step 11   _xxxxxxxxx__C____BA____D   B jumps to 18
step 15   _xxxxxxxxx___xxxCBA____D   C walks to 17
step 16   C's next pad is 18, already used by B  ->  reuse
```

1. **Step 10.** Pad 11 is reserved for B in A’s view, so A jumps.
2. **Step 11.** B has not received A’s jump message. Pad 11 is reserved for A in B’s view, so B jumps too.
3. **Steps 12–16.** C has received A’s jump message but not B’s. C therefore predicts that B walks to 11, and reserves only 11 for B.

Throughout, C has at most 1 undelivered message, B’s jump, at each of its sends.